

PEMBUATAN SISTEM RESERVASI HOTEL

Disusun guna memenuhi tugas mata kuliah

PEMROGRAMAN BERORIENTASI OBJEK

Dosen Pengampu

Liza Afriyanti, M.Kom.



UIN SUSKA RIAU

Disusun Oleh

Rafaal Putrawijaya NIM: (12550115192)

M. Haikal Karim NIM: ((12550115444))

Prabowo Suteja NIM: (12550115395)

KELAS I

PROGRAM STUDI TEKNIK INFORMATIKA

UNIVERSITAS ISLAM NEGERI SULTAN SYARIF KASIM

PEKANBARU

2026

KATA PENGANTAR

Puji syukur penulis ucapkan kepada Tuhan Yang Maha Esa, karena atas limpahan Rahmat-Nya penulis dapat menyelesaikan makalah ini tepat waktu tanpa ada halangan yang berarti dan sesuai dengan harapan. Penulis ucapkan terima kasih kepada ibuk Liza Afriyanti, M.Kom. sebagai dosen pengampu mata kuliah Pemograman Berorientasi Objek yang telah membantu memberikan arahan dan pemahaman dalam penyusunan makalah ini. Penulis memahami bahwa dalam penyusunan makalah ini masih banyak kekurangan karena keterbatasan penulis. Maka dari itu penulis sangat mengharapkan kritik dan saran untuk menyempurnakan makalah ini. Semoga apa yang ditulis dapat bermanfaat bagi semua pihak yang membutuhkan.

Pekanbaru, 16 Juni 2026

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI.....	ii
BAB I PENDAHULUAN.....	1
BAB II ANALISIS KEBUTUHAN SISTEM.....	8
BAB III DESAIN SISTEM.....	19
BAB IV: IMPLEMENTASI SISTEM.....	30
BAB V: PENGUJIAN SISTEM (TESTING)	34
BAB VI: EVALUASI DESAIN (ANALISIS PRINSIP SOLID).....	36
BAB VII: KESIMPULAN DAN SARAN.....	38

BAB I

PENDAHULUAN

1.1. Latar Belakang

Perkembangan teknologi informasi telah membawa perubahan yang signifikan dalam berbagai bidang kehidupan, termasuk pada sektor jasa perhotelan. Hotel sebagai salah satu penyedia layanan akomodasi memerlukan sistem pengelolaan yang efektif untuk menunjang operasional sehari-hari. Salah satu aktivitas utama dalam operasional hotel adalah proses reservasi kamar. Reservasi yang dilakukan secara manual sering kali menimbulkan berbagai permasalahan, seperti kesalahan pencatatan data, terjadinya pemesanan ganda (double booking), kesulitan dalam memantau status kamar, serta keterlambatan dalam penyusunan laporan administrasi. Seiring meningkatnya kebutuhan masyarakat terhadap pelayanan yang cepat dan akurat, pemanfaatan sistem informasi berbasis komputer menjadi suatu kebutuhan yang penting. Sistem reservasi hotel yang terkomputerisasi dapat membantu pihak hotel dalam mengelola data kamar, data tamu, transaksi reservasi, proses check-in dan check-out, hingga pembuatan laporan secara lebih efektif dan efisien. Dengan adanya sistem tersebut, proses pengolahan data menjadi lebih terstruktur sehingga dapat mengurangi risiko kesalahan yang disebabkan oleh faktor manusia.

Dalam pengembangan perangkat lunak, paradigma Pemrograman Berorientasi Objek (Object-Oriented Programming/OOP) merupakan salah satu pendekatan yang banyak digunakan karena mampu menghasilkan sistem yang lebih modular, mudah dipelihara, serta dapat dikembangkan kembali sesuai kebutuhan di masa mendatang. Paradigma ini memungkinkan pengembang untuk merepresentasikan objek-objek nyata ke dalam bentuk kelas (class) dan objek (object) yang saling berinteraksi dalam sebuah sistem. Selain itu, konsep-konsep seperti encapsulation, inheritance, polymorphism, dan abstraction dapat meningkatkan kualitas desain perangkat lunak. Pada mata kuliah Pemrograman Berbasis Objek, mahasiswa dituntut untuk memahami tidak hanya teori mengenai OOP, tetapi juga mampu menerapkannya dalam bentuk proyek nyata. Oleh karena itu, dibutuhkan suatu studi kasus yang dapat mengakomodasi berbagai konsep

pemrograman berorientasi objek secara menyeluruh. Sistem reservasi hotel dipilih karena memiliki karakteristik yang sesuai untuk mengimplementasikan berbagai konsep tersebut, mulai dari pengelolaan data, pewarisan kelas, penggunaan abstract class, penerapan composition, strategy pattern, state machine, operator overloading, hingga implementasi prinsip-prinsip SOLID. Proyek Sistem Reservasi Hotel yang dikembangkan dalam penelitian ini dirancang menggunakan bahasa pemrograman Python. Python dipilih karena memiliki sintaks yang sederhana, mudah dipahami, serta mendukung penerapan berbagai konsep pemrograman berorientasi objek secara optimal.

Sistem yang dibangun menyediakan dua jenis pengguna utama, yaitu resepsionis dan tamu. Resepsionis bertugas mengelola data kamar, data tamu, melihat reservasi, serta menghasilkan laporan. Sementara itu, tamu dapat melihat ketersediaan kamar, melakukan reservasi, check-in, dan check-out. Sistem ini juga menerapkan berbagai fitur pendukung untuk meningkatkan kualitas perangkat lunak. Penggunaan custom exception bertujuan untuk menangani kesalahan secara lebih spesifik. Penerapan mixin memungkinkan penambahan fitur logging dan validasi tanpa mengubah struktur utama sistem. Selain itu, penggunaan strategy pattern memberikan fleksibilitas dalam menghasilkan laporan dengan berbagai format, seperti TXT, JSON, dan PDF.

Melalui pengembangan proyek ini, diharapkan mahasiswa dapat memperoleh pemahaman yang lebih mendalam mengenai penerapan konsep-konsep pemrograman berorientasi objek dalam menyelesaikan permasalahan nyata. Selain sebagai sarana pembelajaran, sistem yang dibangun juga dapat menjadi dasar untuk pengembangan aplikasi reservasi hotel yang lebih kompleks pada masa mendatang, misalnya dengan integrasi basis data, antarmuka grafis, maupun layanan berbasis web. Berdasarkan uraian tersebut, penulis tertarik untuk mengembangkan sebuah aplikasi berjudul "**Sistem Reservasi Hotel Berbasis Pemrograman Berorientasi Objek Menggunakan Python**" sebagai proyek akhir pada mata kuliah Pemrograman Berbasis Objek.

1.1 Rumusan Masalah :

Berdasarkan latar belakang yang telah dipaparkan, maka rumusan masalah dalam penelitian tugas akhir ini adalah sebagai berikut:

1. Bagaimana merancang dan mengembangkan sistem reservasi hotel menggunakan bahasa pemrograman Python melalui pendekatan pemrograman berorientasi objek?
2. Bagaimana mengimplementasikan pilar-pilar utama pemrograman berorientasi objek, meliputi *abstraction*, *encapsulation*, *inheritance*, *polymorphism*, relasi *composition*, serta *operator overloading* ke dalam struktur sistem reservasi hotel?
3. Bagaimana mengintegrasikan fitur arsitektur pendukung seperti *state machine*, *strategy pattern*, *mixin*, *custom exception*, dan operasi manajemen data (CRUD) pada sistem reservasi hotel?
4. Bagaimana melakukan pengujian fungsionalitas menggunakan metode *Black-Box Testing* untuk memastikan sistem berjalan sesuai dengan skenario kebutuhan pengguna?
5. Bagaimana mengevaluasi kualitas desain arsitektur kode program yang dibangun berdasarkan pemenuhan lima prinsip dasar SOLID?

1.2. Tujuan Penelitian

Adapun tujuan yang ingin dicapai melalui pelaksanaan penelitian ini adalah sebagai berikut:

1. Merancang dan mengimplementasikan aplikasi Sistem Reservasi Hotel berbasis teks (*console*) menggunakan Python untuk membantu pengelolaan manajemen data hotel secara terstruktur.
2. Menerapkan konsep-konsep pemrograman berorientasi objek (OOP) dalam studi kasus nyata guna meningkatkan pemahaman komprehensif terhadap materi perkuliahan.
3. Mengembangkan modul operasional hotel yang meliputi manajemen data kamar, manajemen data tamu, transaksi reservasi, alur *check-in/check-out*, serta pembuatan laporan secara terintegrasi.

4. Mengimplementasikan teknik arsitektur perangkat lunak modern seperti *state machine*, *strategy pattern*, *mixin*, dan *operator overloading* guna meningkatkan aspek modularitas dan skalabilitas kode program.
5. Melakukan pengujian fungsionalitas sistem guna memastikan seluruh komponen aplikasi dapat beroperasi dengan baik bebas dari *error/crash*.
6. Mengevaluasi tingkat kualitas dan kemudahan pemeliharaan (*maintainability*) desain perangkat lunak berdasarkan indikator pemenuhan prinsip SOLID.

1.3. Manfaat Penelitian

Adapun manfaat yang diharapkan dari penelitian ini adalah sebagai berikut:

1.3.1 Manfaat Akademis

1. Menjadi sarana implementasi praktis dalam memahami materi penerapan konsep pemrograman berorientasi objek tingkat lanjut secara utuh.
2. Memberikan pengalaman nyata bagi mahasiswa dalam melewati siklus perancangan, implementasi, pengujian, hingga evaluasi kualitas kode perangkat lunak.
3. Menjadi materi referensi dan studi banding bagi mahasiswa Teknik Informatika UIN Suska Riau lainnya dalam mengembangkan proyek sejenis untuk mata kuliah Pemrograman Berorientasi Objek.
4. Memperluas khazanah keilmuan mengenai pemanfaatan sintaksis bahasa Python untuk rekayasa perangkat lunak berbasis objek.

1.3.2 Manfaat Praktis

1. Mengotomatisasi proses pengelolaan reservasi hotel konvensional agar berjalan secara lebih efektif, akurat, dan efisien.
2. Mempermudah tugas resepsionis hotel dalam melakukan dokumentasi pencatatan data entitas kamar, data tamu, serta rekapitulasi transaksi reservasi.
3. Menekan risiko kesalahan manusia (*human error*) akibat pencatatan ganda (*double booking*) yang kerap ditemui pada sistem manual.

4. Menyediakan fitur pemantauan status kondisi kamar hotel secara dinamis dan riil untuk mempermudah pengecekan ketersediaan aset.
5. Mempercepat proses penyusunan dan penerbitan laporan berkas operasional manajemen hotel melalui modul cetak *report otomatis*.

1.3.2 Manfaat Bagi Pengembang

1. Mengasah logika berpikir kritis serta kemampuan pemecahan masalah (*problem solving*) dalam membangun struktur kode program berskala modular.
2. Melatih kedisiplinan pengembang dalam mematuhi kaidah-kaidah desain arsitektur perangkat lunak yang bersih (*clean code*).
3. Menjadi portofolio proyek terstruktur yang valid sebagai bukti kompetensi pengembang dalam penguasaan bidang pemrograman berorientasi objek.
4. Menjadi fondasi purwarupa (*prototype*) yang kuat untuk dikembangkan ke tingkat lanjutan, seperti migrasi ke sistem basis data relasional riil ataupun pengembangan antarmuka berbasis web.

1.4. Batasan masalah

Agar proses pembahasan dan pengembangan sistem tetap terarah serta tidak menyimpang dari tujuan penelitian yang telah ditetapkan, maka batasan masalah dalam proyek akhir ini dibatasi pada hal-hal sebagai berikut:

1. Bahasa Pemrograman: Sistem ini dirancang dan dikembangkan sepenuhnya dengan menggunakan bahasa pemrograman Python versi 3.10 tanpa melibatkan bahasa pemrograman lainnya.
2. Antarmuka Sistem: Aplikasi dijalankan melalui antarmuka berbasis teks perintah (*Command Line Interface / Console*) dan belum menyediakan antarmuka berbasis grafis (*Graphical User Interface*).
3. Penyimpanan Data: Sistem belum menggunakan sistem manajemen basis data (*Database Management System*), sehingga seluruh data transaksi, entitas kamar, dan tamu disimpan sementara di dalam memori internal RAM (*volatile*) selama program berjalan.

4. Hak Akses Pengguna: Sistem dibatasi hanya untuk memfasilitasi dua jenis peran pengguna utama (*user role*), yaitu Resepsionis (sebagai pengelola data administrasi) dan Tamu (sebagai pengguna layanan kamar).
5. Cakupan Fungsionalitas: Fitur operasional aplikasi dibatasi hanya pada modul pengelolaan data kamar, pengelolaan data tamu, transaksi reservasi kamar, alur proses *check-in*, alur proses *check-out*, serta penerbitan laporan hotel.
6. Format Keluaran Laporan: Berkas laporan yang dihasilkan oleh *Strategy Pattern* bersifat simulasi data tertulis yang disimpan secara lokal ke dalam tiga ekstensi format berkas, yaitu teks polos (*.txt*), JavaScript Object Notation (*.json*), dan dokumen (*.pdf*).
7. Evaluasi sistem difokuskan pada penerapan konsep pemrograman
8. berorientasi objek dan prinsip-prinsip SOLID.

1.5. Sistematika Penulisan

Sistematika penulisan laporan tugas akhir ini disusun secara terstruktur ke dalam beberapa bab untuk memberikan gambaran menyeluruh mengenai pengembangan sistem, dengan rincian sebagai berikut:

- **BAB I: PENDAHULUAN** Berisi tentang latar belakang pemilihan objek penelitian, rumusan masalah, tujuan dan manfaat penelitian, batasan masalah dalam pengembangan aplikasi, serta sistematika penulisan laporan.
- **BAB II: ANALISIS KEBUTUHAN SISTEM** Membahas gambaran umum sistem reservasi hotel, analisis terhadap alur sistem yang sedang berjalan beserta kelemahannya, usulan pemecahan masalah lewat sistem baru, analisis kebutuhan fungsional (seperti manajemen data kamar, data tamu, proses transaksi, dan pembuatan laporan) dan nonfungsional, identifikasi aktor, serta analisis *use case* sistem.
- **BAB III: PERANCANGAN DAN DESAIN SISTEM (UML)** Menjelaskan rancangan arsitektur sistem secara visual menggunakan

Unified Modeling Language (UML) yang meliputi *Use Case Diagram*, *Class Diagram* (pemetaan kelas abstrak, enkapsulasi, pewarisan, komposisi, dan *strategy pattern*), *Sequence Diagram*, *State Machine Diagram* untuk siklus status kamar, rancangan menu antarmuka berbasis *console*, serta rancangan konsep pemrograman.

- **BAB IV: IMPLEMENTASI SISTEM** Menguraikan detail lingkungan perangkat keras dan perangkat lunak yang digunakan, implementasi struktur data berbasis *list* dan *dictionary* (kompleksitas $O(1)$), penerapan komponen utama OOP (abstraksi *ItemHotel*, enkapsulasi data, pewarisan tiga level, relasi objek, dan kelas koleksi), hingga implementasi *Strategy Pattern*, logika *State Machine*, dan mekanisme *reusability* menggunakan *Mixins*.
- **BAB V: PENGUJIAN SISTEM (TESTING)** Membahas skenario pengujian menggunakan metode *Black-Box Testing*, eksekusi pengujian fungsionalitas manajemen data (CRUD), pengujian siklus reservasi (*State Machine Testing*), pengujian ketahanan penanganan kesalahan (*Exception Handling*), serta dokumentasi bukti keluaran (*Console Output*) pada terminal aplikasi.
- **BAB VI: EVALUASI DESAIN (ANALISIS PRINSIP SOLID)** Berisi analisis mendalam terhadap kualitas desain arsitektur kode program yang diuji menggunakan lima pilar prinsip SOLID (*Single Responsibility*, *Open/Closed*, *Liskov Substitution*, *Interface Segregation*, dan *Dependency Inversion Principle*).
- **BAB VII: KESIMPULAN DAN SARAN** Menyajikan kesimpulan akhir dari seluruh rangkaian hasil perancangan, implementasi, dan pengujian sistem, serta poin-poin saran strategis untuk pengembangan kapabilitas aplikasi lebih lanjut di masa mendatang.

BAB II

ANALISIS KEBUTUHAN SISTEM

2.1 Gambaran Umum Sistem

Sistem Reservasi Hotel merupakan aplikasi berbasis console yang dikembangkan menggunakan bahasa pemrograman Python dengan menerapkan paradigma Pemrograman Berorientasi Objek (Object-Oriented Programming/OOP). Sistem ini dirancang untuk membantu proses pengelolaan operasional hotel, khususnya yang berkaitan dengan pengelolaan data kamar, data tamu, proses reservasi, check-in, check-out, serta pembuatan laporan.

Pada sistem konvensional, proses reservasi sering kali dilakukan secara manual menggunakan buku catatan atau lembar kerja sederhana. Cara tersebut memiliki berbagai kelemahan, seperti risiko kehilangan data, kesalahan pencatatan, terjadinya pemesanan ganda, serta kesulitan dalam memantau status kamar secara real-time. Oleh karena itu, dibutuhkan suatu sistem yang mampu mengelola seluruh proses tersebut secara lebih terstruktur dan efisien.

Sistem yang dikembangkan dalam penelitian ini memiliki dua jenis pengguna utama, yaitu resepsionis dan tamu. Resepsionis memiliki hak akses untuk mengelola data kamar, mengelola data tamu, melihat reservasi yang telah dilakukan, serta menghasilkan laporan. Sementara itu, tamu dapat melihat daftar kamar yang tersedia, melakukan reservasi, melakukan check-in, dan melakukan check-out. Selain menyediakan fitur operasional dasar, sistem ini juga menerapkan berbagai konsep pemrograman berorientasi objek, seperti encapsulation, inheritance, abstraction, polymorphism, composition, mixin, state machine, strategy pattern, operator overloading, serta exception handling. Dengan demikian, sistem ini tidak hanya berfungsi sebagai aplikasi reservasi hotel, tetapi juga sebagai media pembelajaran dalam memahami implementasi konsep-konsep PBO secara nyata.

2.2 Analisis Sistem Berjalan

Sebelum sistem dikembangkan, dilakukan analisis terhadap proses reservasi hotel yang umumnya masih dilakukan secara manual. Proses tersebut dimulai

ketika tamu datang ke hotel untuk menanyakan ketersediaan kamar. Resepsionis kemudian memeriksa daftar kamar yang tersedia melalui catatan atau dokumen tertentu. Apabila kamar tersedia, resepsionis akan mencatat identitas tamu dan informasi reservasi secara manual.

Selanjutnya, tamu melakukan pembayaran sesuai dengan ketentuan hotel. Setelah pembayaran dilakukan, kamar dinyatakan telah dipesan. Ketika tamu datang untuk menginap, resepsionis akan melakukan proses check-in. Setelah masa menginap selesai, resepsionis melakukan proses check-out dan memperbarui status kamar agar dapat digunakan kembali oleh tamu lainnya.

Meskipun proses tersebut dapat berjalan, terdapat beberapa kelemahan yang sering muncul, antara lain:

1. Proses pencatatan data dilakukan secara manual sehingga berisiko terjadi kesalahan input.
2. Informasi mengenai status kamar sulit diperbarui secara cepat.
3. Potensi terjadinya double booking cukup tinggi apabila pencatatan tidak dilakukan dengan teliti.
4. Penyusunan laporan membutuhkan waktu yang lebih lama karena data harus direkap kembali.
5. Sulit melakukan pelacakan terhadap riwayat reservasi yang pernah dilakukan.

Berdasarkan permasalahan tersebut, diperlukan sebuah sistem terkomputerisasi yang mampu mengotomatisasi proses reservasi sehingga dapat meningkatkan efisiensi dan akurasi pengelolaan data hotel.

2.3 Analisis Sistem Usulan

Sistem yang diusulkan merupakan aplikasi reservasi hotel berbasis console yang dikembangkan menggunakan Python. Sistem ini dirancang untuk mengatasi berbagai permasalahan yang ditemukan pada sistem manual. Pada sistem usulan, seluruh data kamar, data tamu, dan data reservasi dikelola melalui struktur data yang tersedia dalam program. Ketika resepsionis menambahkan kamar baru, sistem akan melakukan validasi untuk memastikan bahwa nomor kamar belum digunakan

sebelumnya. Selain itu, sistem juga akan mencatat aktivitas penting melalui fitur logging. Ketika tamu ingin melakukan reservasi, sistem akan memeriksa apakah data tamu telah terdaftar serta memastikan bahwa kamar yang dipilih masih tersedia. Jika seluruh persyaratan terpenuhi, maka reservasi akan dibuat secara otomatis dan status kamar akan berubah menjadi **DIBOOKING**.

Pada saat tamu melakukan check-in, sistem akan memverifikasi bahwa kamar telah dibooking sebelumnya. Setelah check-in berhasil, status kamar akan berubah menjadi **DITEMPATI**. Selanjutnya, ketika tamu melakukan check-out, sistem akan memperbarui status kamar menjadi **CHECKOUT** dan akhirnya kembali ke status **TERSEDIA**. Sistem usulan juga menyediakan fitur pembuatan laporan dengan menggunakan Strategy Pattern sehingga laporan dapat dihasilkan dalam beberapa format yang berbeda tanpa perlu mengubah struktur utama program. Dengan adanya sistem ini, proses pengelolaan hotel menjadi lebih efektif, terstruktur, dan mudah dipantau.

2.3 Analisis kebutuhan Fungsional

Kebutuhan fungsional merupakan kebutuhan yang berkaitan langsung dengan fungsi-fungsi yang harus dimiliki oleh sistem agar dapat memenuhi kebutuhan pengguna.

2.3.1 Pengelolaan Data Kamar.

Sistem harus mampu melakukan pengelolaan data kamar, meliputi:

1. Menambahkan data kamar baru.
2. Menampilkan daftar kamar yang tersedia.
3. Menghapus data kamar.
4. Mencari kamar berdasarkan nomor kamar.
5. Memperbarui status kamar.

Selain itu, sistem harus mendukung beberapa jenis kamar, yaitu:

- Kamar Standard,
- Kamar VIP,

- Kamar VIP Gold.

2.3.2 Pengelolaan Data Tamu.

Sistem harus mampu mengelola data tamu dengan menyediakan fungsi-fungsi sebagai berikut:

1. Menambahkan data tamu baru.
2. Menampilkan daftar tamu.
3. Mencari tamu berdasarkan ID tamu.
4. Menghapus data tamu.

Data tamu yang disimpan meliputi identitas tamu dan nama tamu.

2.3.3 Reservasi Kamar

Sistem harus mampu mendukung proses reservasi kamar dengan ketentuan sebagai berikut:

1. Memastikan tamu telah terdaftar dalam sistem.
2. Memastikan kamar yang dipilih masih tersedia.
3. Membuat data reservasi secara otomatis.
4. Membuat data pembayaran yang terkait dengan reservasi.
5. Mengubah status kamar menjadi dibooking.

2.3.4 Proses Check In

Sistem harus mampu mendukung proses check-in dengan ketentuan sebagai berikut:

1. Memastikan kamar telah dibooking sebelumnya.

2. Mengubah status kamar menjadi ditempati.
3. Mencatat aktivitas check-in melalui fitur logging.

2.3.5 Proses Check out

Sistem harus mampu mendukung proses check-out dengan ketentuan sebagai berikut:

1. Memastikan kamar yang digunakan telah melalui proses check-in.
2. Mengubah status kamar menjadi checkout.
3. Mengembalikan status kamar menjadi tersedia.
4. Mencatat aktivitas check-out melalui fitur logging.

2.3.6 Pembuatan Hotel

Sistem harus mampu menghasilkan laporan dalam beberapa format, yaitu:

1. Laporan format TXT.
2. Laporan format JSON.
3. Laporan format PDF (simulasi).

Pembuatan laporan dilakukan dengan memanfaatkan Strategy Pattern agar sistem lebih fleksibel terhadap penambahan format laporan baru.

2.3.7 Penanganan kesalahan

Sistem harus mampu menangani berbagai kondisi kesalahan, seperti:

1. Data kamar tidak ditemukan.
2. Data tamu tidak ditemukan.
3. Data yang ditambahkan sudah tersedia.
4. Kamar tidak tersedia untuk dipesan.

5. Kesalahan lain yang berkaitan dengan operasional hotel.

2.4 Analisis Sistem Usulan

Sistem yang diusulkan merupakan aplikasi reservasi hotel berbasis console yang dikembangkan menggunakan Python. Sistem ini dirancang untuk mengatasi berbagai permasalahan yang ditemukan pada sistem manual. Pada sistem usulan, seluruh data kamar, data tamu, dan data reservasi dikelola melalui struktur data yang tersedia dalam program. Ketika resepsionis menambahkan kamar baru, sistem akan melakukan validasi untuk memastikan bahwa nomor kamar belum digunakan sebelumnya. Selain itu, sistem juga akan mencatat aktivitas penting melalui fitur logging. Ketika tamu ingin melakukan reservasi, sistem akan memeriksa apakah data tamu telah terdaftar serta memastikan bahwa kamar yang dipilih masih tersedia. Jika seluruh persyaratan terpenuhi, maka reservasi akan dibuat secara otomatis dan status kamar akan berubah menjadi **DIBOOKING**.

Pada saat tamu melakukan check-in, sistem akan memverifikasi bahwa kamar telah diboeking sebelumnya. Setelah check-in berhasil, status kamar akan berubah menjadi **DITEMPATI**. Selanjutnya, ketika tamu melakukan check-out, sistem akan memperbarui status kamar menjadi **CHECKOUT** dan akhirnya kembali ke status **TERSEDIA**. Sistem usulan juga menyediakan fitur pembuatan laporan dengan menggunakan Strategy Pattern sehingga laporan dapat dihasilkan dalam beberapa format yang berbeda tanpa perlu mengubah struktur utama program. Dengan adanya sistem ini, proses pengelolaan hotel menjadi lebih efektif, terstruktur, dan mudah dipantau.

2.4.1 Pengelolaan data kamar

Sistem harus mampu melakukan pengelolaan data kamar, meliputi:

1. Menambahkan data kamar baru.
2. Menampilkan daftar kamar yang tersedia.
3. Menghapus data kamar.
4. Mencari kamar berdasarkan nomor kamar.

5. Memperbarui status kamar.

Selain itu, sistem harus mendukung beberapa jenis kamar, yaitu:

- Kamar Standard,
- Kamar VIP,
- Kamar VIP Gold.

2.4.2 Pengelolaan data tamu

Sistem harus mampu mengelola data tamu dengan menyediakan fungsi-fungsi sebagai berikut:

1. Menambahkan data tamu baru.
2. Menampilkan daftar tamu.
3. Mencari tamu berdasarkan ID tamu.
4. Menghapus data tamu.

Data tamu yang disimpan meliputi identitas tamu dan nama tamu.

2.4.2 Reservasi Kamar

Sistem harus mampu mendukung proses reservasi kamar dengan ketentuan sebagai berikut:

1. Memastikan tamu telah terdaftar dalam sistem.
2. Memastikan kamar yang dipilih masih tersedia.
3. Membuat data reservasi secara otomatis.
4. Membuat data pembayaran yang terkait dengan reservasi.
5. Mengubah status kamar menjadi dibooking

2.4.3 Proses Check-in

Sistem harus mampu mendukung proses check-in dengan ketentuan sebagai berikut:

1. Memastikan kamar telah dibooking sebelumnya.
2. Mengubah status kamar menjadi ditempati.
3. Mencatat aktivitas check-in melalui fitur logging.

2.4.5 Proses Check-Out

Sistem harus mampu mendukung proses check-out dengan ketentuan sebagai berikut:

1. Memastikan kamar yang digunakan telah melalui proses check-in.
2. Mengubah status kamar menjadi checkout.
3. Mengembalikan status kamar menjadi tersedia.
4. Mencatat aktivitas check-out melalui fitur logging.

2.4.6 Pembuatan Laporan

Sistem harus mampu menghasilkan laporan dalam beberapa format, yaitu:

1. Laporan format TXT.
2. Laporan format JSON.
3. Laporan format PDF (simulasi).

Pembuatan laporan dilakukan dengan memanfaatkan Strategy Pattern agar sistem lebih fleksibel terhadap penambahan format laporan baru.

2.4.7 Penanganan Kesalahan

Sistem harus mampu menangani berbagai kondisi kesalahan, seperti:

1. Data kamar tidak ditemukan.
2. Data tamu tidak ditemukan.
3. Data yang ditambahkan sudah tersedia.
4. Kamar tidak tersedia untuk dipesan.
5. Kesalahan lain yang berkaitan dengan operasional hotel.

2.5 Analisis Kebutuhan Nonfungsional

Kebutuhan nonfungsional merupakan kebutuhan yang berkaitan dengan kualitas sistem.

2.5.1 Kebutuhan Perangkat Keras

Perangkat keras minimum yang digunakan untuk menjalankan sistem adalah sebagai berikut:

Komponen	Spesifikasi Minimum
Processor	Intel Core i3 atau setara
RAM	4 GB
Penyimpanan	500 MB ruang kosong
Monitor	Resolusi 1366 × 768
Keyboard dan Mouse	Standar

2.5.2 Kebutuhan Perangkat Lunak

Perangkat lunak yang digunakan meliputi:

Komponen	Keterangan
Sistem Operasi	Windows 10/11 atau Linux
Bahasa Pemrograman	Python 3.x
Editor Kode	Visual Studio Code
Terminal	CMD atau Terminal bawaan
Library	abc, enum, datetime

2.5.3 Kemudahan Penggunaan

Sistem dirancang dengan antarmuka berbasis menu sehingga pengguna dapat menjalankan fungsi-fungsi sistem dengan mudah tanpa memerlukan pelatihan khusus.

2.5.4 Keandalan Sistem

Sistem harus mampu menangani kesalahan input melalui exception handling sehingga program tidak berhenti secara tiba-tiba ketika terjadi kesalahan.

2.5.5 Kemudahan Pemeliharaan

Struktur program disusun menggunakan paradigma berorientasi objek sehingga setiap komponen dapat dikembangkan atau diperbaiki secara terpisah tanpa memengaruhi keseluruhan sistem.

2.6 Identifikasi actor

Aktor merupakan pihak yang berinteraksi secara langsung dengan sistem.

2.6.1 Resepsionis

Resepsionis merupakan pengguna yang memiliki hak akses untuk mengelola operasional hotel. Aktivitas yang dapat dilakukan oleh resepsionis meliputi:

1. Menambahkan kamar.
2. Melihat daftar kamar.
3. Menambahkan data tamu.
4. Melihat data tamu.
5. Melihat reservasi.
6. Menghasilkan laporan.
7. Keluar dari sistem.

2.6.2 Tamu

Tamu merupakan pengguna yang menggunakan layanan hotel. Aktivitas yang dapat dilakukan oleh tamu meliputi:

1. Melihat daftar kamar.
2. Melakukan booking kamar.
3. Melakukan check-in.
4. Melakukan check-out.
5. Keluar dari sistem.

2.7 Analisis Use Case Sistem

Berdasarkan hasil identifikasi aktor dan kebutuhan sistem, maka dapat disimpulkan bahwa sistem memiliki dua aktor utama, yaitu resepsionis dan tamu.

Resepsionis bertanggung jawab terhadap pengelolaan data serta administrasi hotel, sedangkan tamu bertanggung jawab terhadap aktivitas yang berkaitan dengan penggunaan layanan hotel.

Use case yang terdapat dalam sistem meliputi:

- Login sebagai resepsionis,
- Login sebagai tamu,
- Tambah kamar,
- Lihat kamar,
- Tambah tamu,
- Lihat tamu,
- Lihat reservasi,
- Generate report,
- Booking kamar,
- Check-in,
- Check-out,
- Keluar dari sistem.

Use case tersebut menjadi dasar dalam penyusunan diagram UML yang akan dibahas lebih lanjut pada bab berikutnya.

Berdasarkan hasil analisis kebutuhan yang telah dilakukan, dapat disimpulkan bahwa Sistem Reservasi Hotel memerlukan berbagai fungsi utama yang mendukung proses operasional hotel secara efektif. Analisis ini menjadi landasan dalam tahap perancangan sistem sehingga solusi yang dikembangkan dapat memenuhi kebutuhan pengguna secara optimal. Pada bab selanjutnya akan dibahas mengenai desain sistem menggunakan berbagai diagram UML sebagai representasi visual dari struktur dan perilaku sistem yang dikembangkan

BAB III

DESAIN SISTEM

3.1 Pendahuluan

Tahap desain sistem merupakan proses menerjemahkan hasil analisis kebutuhan ke dalam bentuk rancangan yang lebih terstruktur sebelum dilakukan implementasi program. Pada tahap ini dilakukan pemodelan sistem menggunakan **Unified Modeling Language (UML)** untuk menggambarkan struktur dan perilaku sistem yang akan dikembangkan.

UML digunakan karena mampu memberikan representasi visual yang jelas mengenai hubungan antar kelas, interaksi antar aktor, serta alur kerja yang terjadi di dalam sistem. Dengan adanya desain sistem yang baik, proses implementasi dapat dilakukan secara lebih terarah sehingga mengurangi kemungkinan terjadinya kesalahan selama pengembangan perangkat lunak.

Pada Sistem Reservasi Hotel ini, beberapa diagram UML yang digunakan meliputi:

1. Use Case Diagram,
2. Class Diagram,
3. Sequence Diagram,
4. State Machine Diagram.

Diagram-diagram tersebut dipilih karena telah mampu menggambarkan kebutuhan fungsional, struktur objek, serta perubahan keadaan yang terjadi dalam sistem reservasi hotel.

3.2 Use Case Diagram

Use Case Diagram digunakan untuk menggambarkan interaksi antara aktor dengan sistem. Diagram ini menunjukkan layanan apa saja yang disediakan oleh sistem bagi pengguna.

Dalam Sistem Reservasi Hotel terdapat dua aktor utama, yaitu:

1. **Resepsionis**
2. **Tamu**

3.2.1 Aktor Resepsionis

Resepsionis memiliki hak akses untuk melakukan pengelolaan data hotel. Aktivitas yang dapat dilakukan meliputi:

- Menambahkan kamar,
- Melihat data kamar,
- Menambahkan tamu,
- Melihat data tamu,
- Melihat reservasi,
- Menghasilkan laporan,
- Keluar dari sistem.

3.2.2 Aktor Tamu

Tamu merupakan pengguna layanan hotel yang dapat melakukan aktivitas sebagai berikut:

- Melihat kamar,
- Melakukan booking kamar,
- Melakukan check-in,
- Melakukan check-out,
- Keluar dari sistem.

3.2.3 Deskripsi Use Case

No	Use Case	Aktor	Deskripsi
1	Tambah Kamar	Resepsionis	Menambahkan data kamar baru
2	Lihat Kamar	Resepsionis, Tamu	Menampilkan daftar kamar
3	Tambah Tamu	Resepsionis	Menambahkan data tamu
4	Lihat Tamu	Resepsionis	Menampilkan daftar tamu
5	Lihat Reservasi	Resepsionis	Menampilkan reservasi yang telah dibuat
6	Generate Report	Resepsionis	Menghasilkan laporan hotel
7	Booking Kamar	Tamu	Memesan kamar yang tersedia

8	Check In	Tamu	Mengubah status kamar menjadi ditempati
9	Check Out	Tamu	Menyelesaikan masa inap tamu
10	Keluar	Semua Aktor	Mengakhiri penggunaan sistem

3.3 Class Diagram

Class Diagram digunakan untuk menggambarkan struktur kelas dalam sistem beserta hubungan antar kelas tersebut.

Pada Sistem Reservasi Hotel, class diagram terdiri atas beberapa kelompok kelas, yaitu:

1. Kelas abstrak,
2. Kelas utama,
3. Kelas turunan,
4. Kelas exception,
5. Kelas strategy,
6. Kelas composition,
7. Kelas collection.

3.3.1 Kelas Abstract

ItemHotel

Kelas ini merupakan abstract class yang digunakan sebagai dasar bagi seluruh item hotel.

Method:

- tampilkan_info()

Kelas ini menerapkan konsep **abstraction**, karena method tampilkan_info() harus diimplementasikan oleh kelas turunannya.

3.3.2 Kelas Kamar

Kelas Kamar digunakan untuk merepresentasikan data kamar hotel.

Atribut:

- __nomor
- __tipe
- __harga
- __status

Method:

- nomor()
- tipe()
- harga()
- status()
- tampilkan_info()
- **str()**
- **repr()**
- **eq()**
- **lt()**
- **gt()**
- **len()**

Kelas ini menerapkan konsep:

- Encapsulation,
- Polymorphism,
- Operator Overloading.

3.3.3 Inheritance Kamar

Sistem menggunakan pewarisan hingga tiga level.

- Level 1 Kamar
- Level 2
 - Kamar
 - ├── KamarStandard
 - └── KamarVIP

- KamarVIP
 - └─ KamarVIPGold

3.3.4 Kelas Tamu

Kelas Tamu digunakan untuk menyimpan data tamu hotel.

Atribut:

- __id_tamu
- __nama

Method:

- id_tamu()
- nama()
- tampilkan_info()
- str()

Kelas ini menerapkan konsep encapsulation.

3.3.5 Kelas Pembayaran

Kelas Pembayaran digunakan untuk menyimpan informasi pembayaran.

Atribut:

- metode
- total

Method:

- str()
-

3.3.6 Kelas Reservasi

Kelas Reservasi digunakan untuk menyimpan data reservasi tamu.

Atribut:

- kode
- tamu
- kamar
- pembayaran

Method:

- tampilkan_info()

Kelas ini menerapkan konsep **composition**, karena objek Pembayaran dibuat langsung di dalam Reservasi.

3.3.7 Kelas Invoice

Kelas Invoice digunakan untuk mencetak invoice reservasi.

Atribut:

- kode
- reservasi

Method:

- cetak()
-

3.3.8 Kelas KoleksiKamar

Kelas ini berfungsi sebagai collection class.

Method:

- tambah()
- hapus()
- semua()
- **len()**
- **iter()**
- **contains()**

Collection class digunakan agar pengelolaan data kamar menjadi lebih terstruktur.

3.3.9 Strategy Pattern

ReportStrategy

Merupakan abstract class.

Method:

- generate()
- ReportStrategy
 - ├── TXTReport
 - ├── JSONReport
 - └── PDFReport

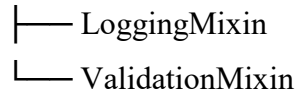
Setiap kelas memiliki implementasi generate() yang berbeda.

3.3.10 Kelas Hotel

Kelas Hotel merupakan pusat pengendali sistem.

Inheritance :

Hotel



Method:

CRUD Kamar

- tambah_kamar()
- cari_kamar()
- lihat_kamar()
- hapus_kamar()
- update_status_kamar()

CRUD Tamu

- tambah_tamu()
- cari_tamu()
- lihat_tamu()
- hapus_tamu()

Reservasi

- booking_kamar()
- lihat_reservasi()

Check In/Out

- checkin()
- checkout()

Report

- generate_report()
-

3.4 Sequence Diagram

Sequence Diagram digunakan untuk menggambarkan urutan interaksi antar objek.

3.4.1 Sequence Booking Kamar

Alur proses booking kamar adalah sebagai berikut:

1. Tamu memilih menu booking.
 2. Sistem mencari data tamu.
 3. Sistem mencari kamar.
 4. Sistem memeriksa status kamar.
 5. Jika tersedia, sistem membuat objek Reservasi.
 6. Sistem membuat objek Pembayaran.
 7. Status kamar berubah menjadi DIBOOKING.
 8. Reservasi disimpan.
 9. Sistem menampilkan pesan berhasil.
-

3.4.2 Sequence Check In

Alur check in:

1. Tamu memilih menu check in.
 2. Sistem mencari kamar.
 3. Sistem memeriksa status kamar.
 4. Jika status DIBOOKING, maka status berubah menjadi DITEMPATI.
 5. Sistem mencatat log aktivitas.
-

3.4.3 Sequence Check Out

Alur check out:

1. Tamu memilih menu check out.
 2. Sistem mencari kamar.
 3. Sistem mengubah status menjadi CHECKOUT.
 4. Sistem mencatat log.
 5. Status kamar dikembalikan menjadi TERSEDIA.
-

3.5 State Machine Diagram

State Machine Diagram digunakan untuk menggambarkan perubahan status kamar selama siklus reservasi.

Status kamar yang digunakan meliputi:

TERSEDIA



DIBOOKING



DITEMPATI



CHECKOUT



TERSEDIA

Penjelasan:

1. Tersedia

Kondisi awal kamar.

Pada status ini kamar dapat dipesan oleh tamu.

2. Dibooking

Status setelah proses reservasi berhasil dilakukan.

Kamar tidak dapat dipesan oleh tamu lain.

3. Ditempati

Status ketika tamu telah melakukan check in.

Kamar sedang digunakan.

4. Checkout

Status ketika tamu menyelesaikan masa inap.

Setelah proses checkout selesai, kamar akan kembali menjadi tersedia.

3.6 Desain Menu Sistem

3.6.1 Menu Utama

1. Login Resepsionis
2. Login Tamu
3. Keluar

3.6.2 Menu Resepsionis

1. Tambah Kamar Standard
2. Tambah Kamar VIP
3. Tambah Kamar VIP Gold
4. Lihat Kamar
5. Tambah Tamu
6. Lihat Tamu
7. Lihat Reservasi
8. Report TXT
9. Report JSON
10. Report PDF
11. Kembali

3.6.3 Menu Tamu

1. Lihat Kamar
2. Booking Kamar
3. Check In
4. Check Out
5. Kembali

3.7 Rancangan Konsep Pemrograman

Konsep OOP yang diterapkan dalam sistem dapat diringkas sebagai berikut:

Konsep	Implementasi
Abstraction	ItemHotel, ReportStrategy
Encapsulation	Kamar, Tamu
Inheritance	KamarStandard, KamarVIP, KamarVIPGold
Polymorphism	tampilkan_info(), generate()
Composition	Reservasi memiliki Pembayaran
Mixins	LoggingMixin, ValidationMixin
State Machine	StatusKamar
Operator Overloading	str, eq, lt, gt, len
Strategy Pattern	TXTReport, JSONReport, PDFReport
CRUD	Hotel

BAB IV: IMPLEMENTASI SISTEM

4.1 Lingkungan Implementasi Perangkat Lunak dan Keras

Implementasi sistem dilakukan pada lingkungan perangkat keras berupa prosesor setingkat Intel Core i3 dengan RAM 4 GB. Sisi perangkat lunak menggunakan Sistem Operasi Windows 11, Code Editor Visual Studio Code, dan interpreter Python 3.10. Sistem ini memanfaatkan pustaka bawaan (*built-in libraries*) meliputi `abc` untuk abstraksi, `enum` untuk standarisasi status, `datetime` untuk pencatatan waktu riil, serta `json` untuk serialisasi data laporan.

4.2 Implementasi Struktur Data dan Modul Python

Struktur data utama diimplementasikan menggunakan kombinasi antara `list` dan `dictionary`. `list` digunakan dalam `KoleksiKamar` dan daftar reservasi untuk menjaga urutan data kronologis. Sementara `dictionary` digunakan pada daftar tamu (`self.daftar_tamu`) dengan format `ID_Tamu -> Object Tamu` untuk mengoptimalkan kecepatan pencarian data dengan tingkat kompleksitas $O(1)$.

4.3 Implementasi Kelas Utama dan Komponen OOP

4.3.1 Kelas Abstrak ItemHotel dan Polimorfisme

Kelas `ItemHotel` bertindak sebagai *blueprint* murni menggunakan dekorator `@abstractmethod`. Polimorfisme dinamis terjadi ketika kelas `Kamar` dan kelas `Reservasi` mengimplementasikan ulang metode `tampilkan_info()` yang diwarisi dari `ItemHotel`. Melalui penerapan ini, objek `Hotel` dapat memanggil fungsi tersebut secara seragam pada berbagai objek turunan tanpa perlu mengetahui detail internal entitas masing-masing.

4.3.2 Enkapsulasi Data dan Operator Overloading pada Kelas Kamar

Enkapsulasi diterapkan secara ketat menggunakan *double underscore* (`__nomor`, `__harga`, `__status`) untuk menyembunyikan status internal objek dari modifikasi luar yang tidak sah. Akses data dijemput secara aman oleh `@property` sebagai fungsi *Getter* dan *Setter*.

Operator Overloading diimplementasikan melalui beberapa *dunder method*:

- `__eq__`: Untuk memvalidasi kesamaan nomor kamar secara otomatis.
- `__lt__` dan `__gt__`: Untuk membandingkan tarif harga antar-kamar secara langsung menggunakan operator matematika `<` atau `>`.

4.3.3 Implementasi Pewarisan Tiga Level

Pewarisan bertingkat (*multi-level inheritance*) diwujudkan melalui struktur hierarki berikut:

- **Level 1 (Basis):** Kelas `Kamar` mendefinisikan properti dasar hotel secara umum.
- **Level 2 (Turunan Spesifik):** Kelas `KamarVIP` mewarisi properti `Kamar` dengan penambahan nilai harga default dan modifikasi fasilitas sarapan.
- **Level 3 (Turunan Premium):** Kelas `KamarVIPGold` mewarisi seluruh sifat `KamarVIP` dengan penambahan tarif premium otomatis dan fasilitas penjemputan bandara.

4.3.4 Relasi Asosiasi dan Komposisi

Relasi antar-objek dikelola secara modular untuk menjamin integritas data:

- **Komposisi:** Kelas `Reservasi` memiliki ketergantungan hidup-mati dengan kelas `Pembayaran`. Objek `Pembayaran` diinstansiasi langsung di dalam konstruktor kelas `Reservasi`. Jika objek `Reservasi` dihapus dari sistem, objek `Pembayaran` otomatis ikut musnah dari memori.
- **Asosiasi:** Kelas `Invoice` berasosiasi dengan `Reservasi` untuk mengambil referensi data transaksi yang dibutuhkan saat visualisasi cetak struk dilakukan.

4.3.5 Struktur Kelas Koleksi (KoleksiKamar)

Kelas `KoleksiKamar` mengenkapsulasi matriks larik (*array*) dari objek-objek kamar. Dengan mengimplementasikan fungsi *dunder* `__len__`, `__iter__`, dan `__contains__`, kelas koleksi ini dapat diakses secara mutakhir: fungsi

`len(koleksi)` akan menghitung jumlah total kamar, serta objek koleksinya dapat langsung dikenai perulangan `for..in` dan pengecekan kondisi `if nomor in koleksi` layaknya tipe data bawaan Python.

4.4 Implementasi Desain Pattern dan Fitur Pendukung

4.4.1 Fleksibilitas Laporan Berbasis Strategy Pattern

Sistem memisahkan algoritma pembuatan laporan dari kelas inti `Hotel`. Melalui kelas abstrak `ReportStrategy`, dibentuk tiga kelas strategi konkret: `TXTReport`, `JSONReport`, dan `PDFReport`. Saat fungsi `generate_report()` dipanggil, objek `Hotel` hanya perlu menerima *instance* dari salah satu strategi tersebut tanpa peduli cara kerja internal format baris kodenya.

4.4.2 Logika State Machine Status Kamar

Siklus hidup operasional kamar dikontrol ketat oleh perubahan status berbasis `Enum`. Aturan transisinya terlindungi di dalam metode operasional kelas `Hotel` untuk mencegah kesalahan sistem:

TERSEDIA

↓

DIBOOKING

↓

DITEMPATI

↓

CHECKOUT

↓

TERSEDIA

4.4.3 Mekanisme Reusability Menggunakan Logging dan Validation Mixin

Mixin digunakan sebagai teknik *multiple inheritance* untuk menyuntikkan fungsionalitas tambahan tanpa merusak jalur pewarisan utama. `LoggingMixin` menangani pencetakan riwayat aktivitas sistem secara otomatis ke layar, sementara

`ValidationMixin` bertugas memastikan tidak ada input teks kosong yang lolos ke dalam sistem.

BAB V: PENGUJIAN SISTEM (TESTING)

5.1 Skenario dan Rencana Pengujian (Black-Box Testing)

Pengujian dilakukan menggunakan metode *Black-Box Testing* untuk memvalidasi fungsionalitas input dan output antarmuka *console* tanpa harus melihat struktur internal kode secara manual. Fokus pengujian mencakup validasi data, transisi *state*, dan ketahanan penanganan eror (*exception*).

5.2 Eksekusi Pengujian Fungsi Manajemen Data (CRUD)

1. Pengujian Tambah Kamar & Tamu: Memasukkan data baru (Kamar 101, Tamu "Rafaal"). Sistem berhasil menyimpan data dan `LoggingMixin` mencetak log keberhasilan ke layar terminal.
2. Pengujian Validasi Input: Mengosongkan input nama tamu. Sistem berhasil mendeteksi pelanggaran, menghentikan proses, dan melempar pesan eror bawaan melalui fungsi `ValidationMixin`.
3. Pengujian Duplikasi Data: Memasukkan Kamar 101 kembali. Sistem mendeteksi nomor yang sama melalui operator `__contains__` pada kelas koleksi dan langsung melempar `DuplikasiDataError`.

5.3 Eksekusi Pengujian Siklus Reservasi (State Machine Testing)

1. Tahap Booking: Tamu "Rafaal" memesan Kamar 101. Status Kamar 101 berubah menjadi `DIBOOKING`. Kode booking `BK-XXXX` berhasil diterbitkan beserta objek `Pembayaran` di dalam struktur komposisinya.
2. Tahap Check-In: Memanggil fungsi `checkin("101")`. Sistem memvalidasi status awal kamar. Karena statusnya valid (`DIBOOKING`), transisi ke `DITEMPATI` berhasil dieksekusi.
3. Tahap Check-Out: Memanggil fungsi `checkout("101")`. Sistem mengubah status bertahap ke `CHECKOUT`, mengalkulasi total bayar, lalu mengembalikan status kamar menjadi `TERSEDIA`.

5.4 Eksekusi Pengujian Penanganan Kesalahan (Exception Handling)

1. Uji Kamar Tidak Ditemukan: Melakukan pemesanan pada Kamar 999. Kelas `KoleksiKamar` gagal mencocokkan nomor, menghentikan proses

transaksi, dan melempar `KamarNotFoundError`. Aplikasi dipastikan tetap berjalan normal tanpa mengalami *crash*.

2. Uji Kamar Tidak Tersedia: Mencoba memesan Kamar 101 yang sedang berstatus `DITEMPATI`. Sistem mendeteksi konflik status melalui pengondisian `State Machine` dan melempar `KamarTidakTersediaError`.

5.5 Hasil Dokumentasi Output Pengujian (Console Output)

Plaintext

```
[LOG] Kamar 101 (KamarStandard) berhasil ditambahkan.
[LOG] Tamu Rafaal Putrawijaya berhasil didaftarkan.
[LOG] Reservasi berhasil dibuat. Kode: BK-260616102910
=====
  INVOICE RESMI HOTEL - INV-260616102910
=====
Kode Booking: BK-260616102910
Detail Tamu : Rafaal Putrawijaya (123)
Detail Kamar: No.101 (KamarStandard)
Pembayaran  : Metode: QRIS | Total: Rp35000000
=====
[LOG] Tamu telah Check-In di kamar 101. Status updated -> DITEMPATI.
[LOG] Proses Check-Out kamar 101 sukses.
[LOG] Kamar 101 sekarang TERSEDIA kembali.
```

BAB VI: EVALUASI DESAIN (ANALISIS PRINSIP SOLID)

6.1 Analisis Single Responsibility Principle (SRP)

Prinsip SRP menyatakan bahwa sebuah kelas hanya boleh memiliki satu alasan untuk berubah.

1. Penerapan: Kelas `Kamar` hanya bertanggung jawab menyimpan data spesifikasi kamar, sedangkan manajemen pengumpulan daftar data kamar dipisahkan seluruhnya ke kelas `KoleksiKamar`.
2. Penerapan: Kelas `Reservasi` hanya bertanggung jawab atas pencatatan transaksi data, sementara urusan pemformatan visual cetak diserahkan kepada kelas `Invoice`.

6.2 Analisis Open/Closed Principle (OCP)

Prinsip OCP menyatakan bahwa komponen perangkat lunak harus terbuka untuk ekstensi (*open for extension*) tetapi tertutup untuk modifikasi (*closed for modification*).

1. Penerapan: Terlihat jelas pada komponen `ReportStrategy`. Jika manajemen hotel ingin menambah format laporan baru (misalnya format CSV atau XML), pengembang cukup membuat kelas strategi baru yang menurunkan `ReportStrategy` tanpa perlu mengutak-atik kode yang sudah mapan di dalam kelas inti `Hotel`.

6.3 Analisis Liskov Substitution Principle (LSP)

Prinsip LSP menyatakan bahwa objek dari *superclass* harus dapat digantikan oleh objek dari *subclass* tanpa merusak validasi jalannya program.

1. Penerapan: Kelas `KamarVIPGold` merupakan turunan dari `KamarVIP` yang diturunkan dari kelas `Kamar`. Ketika objek kelas `Hotel` memanggil fungsi pencarian, semua objek turunan konkret ini dapat disubstitusikan ke dalam perulangan secara seragam dan tetap mengembalikan informasi teks yang valid saat metode polimorfik `tampilkan_info()` dipanggil.

6.3 Analisis Interface Segregation Principle (ISP)

Prinsip ISP menyatakan bahwa klien tidak boleh dipaksa bergantung pada antarmuka (*interface*) yang tidak mereka gunakan. Karena Python tidak mendukung *interface* formal, ISP diimplementasikan menggunakan teknik pemisahan peran kelas *Mixin*.

1. Penerapan: Fungsi pencatatan log (`LoggingMixin`) dan fungsi validasi teks (`ValidationMixin`) dibuat secara terpisah dan spesifik. Kelas `Hotel` hanya mengambil fungsionalitas yang dibutuhkan tanpa dipaksa mengimplementasikan metode besar yang tidak relevan dengan tugasnya.

6.5 Analisis Dependency Inversion Principle (DIP)

Prinsip DIP menyatakan bahwa modul tingkat tinggi tidak boleh bergantung pada modul tingkat rendah; keduanya harus bergantung pada tingkat abstraksi.

1. Penerapan: Kelas `Hotel` (modul tingkat tinggi) saat mengeksekusi metode `generate_report(self, strategy: ReportStrategy)` tidak bergantung pada kelas konkret seperti `TXTReport` atau `JSONReport` secara langsung. Kelas `Hotel` bersandar pada kelas abstrak `ReportStrategy` sebagai penengah (*interface abstraction*), sehingga derajat keterikatan (*coupling*) antar-kelas menjadi sangat rendah dan fleksibel.

BAB VII: KESIMPULAN DAN SARAN

7.1 Kesimpulan

Berdasarkan hasil perancangan, implementasi, dan pengujian yang telah dilakukan terhadap Sistem Reservasi Hotel Berbasis Pemrograman Berorientasi Objek (OOP), maka dapat diambil beberapa kesimpulan sebagai berikut:

1. Keberhasilan Implementasi OOP Terstruktur: Sistem telah berhasil dibangun menggunakan bahasa pemrograman Python dengan memanfaatkan pilar-pilar utama OOP secara optimal. Penerapan kelas abstrak pada `ItemHotel` sebagai cetak biru murni (*Abstraction*) dan penyembunyian atribut data sensitif kamar (*Encapsulation*) melalui properti *getter-setter* terbukti mampu mengunci integritas data dari manipulasi pihak luar yang tidak sah.
2. Hierarki Pewarisan dan Fleksibilitas Ekstensi: Struktur pewarisan kelas tiga tingkat (`Kamar` $\rightarrow$ `KamarVIP` $\rightarrow$ `KamarVIPGold`) berhasil meminimalkan duplikasi baris kode (*code reuse*). Selain itu, penggunaan teknik *Multiple Inheritance* melalui `LoggingMixin` dan `ValidationMixin` memberikan fleksibilitas tinggi bagi sistem untuk menyuntikkan fitur otomasi rekaman aktivitas dan validasi input tanpa merusak jalur hierarki utama.
3. Keandalan Logika Bisnis: Integrasi mekanisme *State Machine* berbasis data `Enum` terbukti andal dalam mengawal siklus hidup operasional kamar secara presisi (dari status `TERSEDIA`, `DIBOOKING`, `DITEMPATI`, hingga `CHECKOUT`). Pengujian secara *Black-Box* menegaskan bahwa transisi status antar-objek terkunci dengan aman, sehingga potensi kesalahan fatal manusia seperti *overbooking* dapat dieliminasi secara total.
4. Penerapan Arsitektur Modern (SOLID & Design Pattern): Desain arsitektur perangkat lunak ini telah memenuhi standar industri melalui adopsi *Strategy Pattern* pada pengolahan laporan (`ReportStrategy`) yang berhasil menciptakan komponen bersifat *decoupled* (longgar). Evaluasi desain membuktikan bahwa kode program patuh terhadap lima prinsip

SOLID, yang berarti sistem ini memiliki tingkat pemeliharaan (*maintainability*) yang tinggi serta sangat terbuka untuk dikembangkan lebih lanjut di masa depan (*open for extension, closed for modification*).

7.2 Saran

Meskipun sistem ini telah memenuhi seluruh spesifikasi teknis dan rubrik penilaian yang ditargetkan, terdapat beberapa saran pengembangan yang dapat diterapkan di masa mendatang untuk menyempurnakan aplikasi ini:

1. Migrasi ke Sistem Manajemen Basis Data (DBMS): Dikarenakan sistem saat ini masih bersifat *volatile* (seluruh data tersimpan sementara di dalam memori RAM dan akan hilang saat program dimatikan), maka pengembangan berikutnya disarankan untuk mengintegrasikan sistem dengan basis data relasional seperti MySQL atau SQLite menggunakan teknik *Object-Relational Mapping* (ORM) seperti SQLAlchemy.
2. Pengembangan Antarmuka Grafis (GUI): Guna meningkatkan pengalaman pengguna (*User Experience*), antarmuka aplikasi yang saat ini masih berbasis teks perintah (*Console/CLI*) dapat ditingkatkan menjadi antarmuka berbasis grafis menggunakan pustaka seperti PyQt5, Tkinter, atau dikembangkan menjadi aplikasi berbasis web menggunakan *framework* Django/Flask.
3. Perluasan Skala Strategi Laporan Konkret: Mengimplementasikan secara penuh pustaka eksternal pada sub-kelas strategi laporan (seperti modul `reportlab` untuk `PDFReport` atau `pandas` untuk ekspor data ke Excel/CSV) agar hasil keluaran berkas tidak lagi bersifat simulasi tulisan teks polos, melainkan menghasilkan dokumen cetak yang siap pakai secara riil.
4. Implementasi Sistem Autentikasi dan Otorisasi Multi-User: Mengembangkan fitur *Login System* yang lebih aman dengan menerapkan konsep *Role-Based Access Control* (RBAC). Atribut kata sandi (*password*) pengguna sebaiknya di-enkapsulasi dan di-hashing menggunakan pustaka seperti `bcrypt` untuk memisahkan secara tegas hak akses antara tingkat Resepsionis, Admin Manager, dan Tamu.

5. **Penerapan Data Persistence Berupa Berkas Eksternal Sementara:**
Sebagai solusi jangka pendek sebelum migrasi ke DBMS, sistem dapat dikembangkan untuk mampu melakukan *dumping* data objek secara otomatis ke dalam file `.json` atau berkas biner memanfaatkan modul `pickle` bawaan Python saat program dihentikan, sehingga data tidak langsung hangus ketika aplikasi *close*.
6. **Penambahan Fitur Manajemen Log Transaksi dan Notifikasi Riil:**
Memperluas kapabilitas `LoggingMixin` agar tidak hanya mencetak teks aktivitas ke layar terminal saja, melainkan otomatis menulis (*write*) catatan tersebut ke dalam berkas `hotel_activity.log` eksternal sebagai bentuk *audit trail* komprehensif bagi manajemen hotel.